Advanced Topics in Algorithms

written by Ron Goldman
based on lectures given by Prof. Haim Kaplan

November 11, 2026

Contents

1 Negative-weight single-source shortest paths **1**
1.1 The classical algorithms . . . 2
1.2 Combining Bellman-Ford and Dijkstra . . . 3
1.3 Potentials . . . 4
1.4 Restricted graphs . . . 4
1.5 Scaling: from the general case to restricted graphs . . . 4
1.6 Directed low-diameter decomposition . . . 5

2 Restricted SSSP **6**
2.1 The crucial property of the LDD . . . 6
2.2 Correctness . . . 6
2.3 Running time and Las Vegas . . . 8

3 Low-diameter decomposition **8**
3.1 Undirected graphs . . . 8
3.2 Directed graphs . . . 9
3.3 Running time and heavy-light classification . . . 9

4 A strongly polynomial algorithm for dense graphs (LLRZ) **10**
4.1 Setup and main idea . . . 10
4.2 3-to-2 shortcutting . . . 11
4.3 How many shortcuts do we add? . . . 11
4.4 Balancing the searches . . . 12

1 Negative-weight single-source shortest paths

We are given a directed graph $G = (V, E, w)$ with integer edge weights, possibly negative, and a source s. Two recent results:

- Bernstein, Nanongkai, Wulff-Nilsen: Negative-weight single-source shortest paths in near-linear time (FOCS 2022, JACM 2025).
- Bringmann, Cassis, Fischer: Negative-weight single-source shortest paths in near-linear time: now faster! (FOCS 2023).
- There are further follow-ups by now, also deterministic ones.

Theorem 1 (Bringmann et al.)**.** *There is a Las Vegas algorithm which, given a directed graph G with integer weights $w(e) \geq -3W$ and a source s, either computes a shortest path tree from s or finds a negative cycle in G, in expected time $O(\text{Dijkstra}(G) \cdot \log^2 n \cdot \log(nW))$, where $\text{Dijkstra}(G) = O(m + n \log n)$.*

In these lectures we prove a slightly weaker version. The presentation follows Fischer et al., "A simple parallel algorithm with near-linear work for negative-weight single-source shortest paths" (SOSA 2025).

Theorem 2. *There is a Las Vegas algorithm which, given a directed graph G without negative cycles, with integer weights $w(e) \geq -3W$, and a source s, computes a shortest path tree from s in expected time $O(\text{Dijkstra}(G) \cdot \log^5 n \cdot \log(nW))$.*

1.1 The classical algorithms

- Dijkstra: only for nonnegative weights; $O(m + n \log n)$ with Fibonacci heaps.
- Bellman-Ford: general weights, detects a negative cycle; $O(mn)$. It can stop early if the shortest paths contain few edges.

Algorithm 1 Dijkstra

```
1:  function DIJKSTRA(G, s)
2:      d_v ← ∞ for all v ∈ V; d_s ← 0
3:      Q ← PriorityQueue({d_s : s})
4:      while Q ≠ ∅ do
5:          pop u ∈ Q with minimal d_u
6:          for each (u, v) ∈ E do
7:              if d_u + w(u, v) < d_v then
8:                  d_v ← d_u + w(u, v); add/update (d_v : v) in Q
9:              end if
10:         end for
11:     end while
12: end function
```

Algorithm 2 Bellman-Ford

```
1: function BELLMAN-FORD(G, s)
2:     d_v ← ∞ for all v ∈ V; d_s ← 0
3:     for i = 1, ..., n − 1 do
4:         for each (u, v) ∈ E do
5:             if d_u + w(u, v) < d_v then d_v ← d_u + w(u, v)
6:             end if
7:         end for
8:     end for
9: end function
```

1.2 Combining Bellman-Ford and Dijkstra

The algorithm alternates a Dijkstra phase on the nonnegative edges with a Bellman-Ford phase on the negative edges.

Algorithm 3 Bellman-Ford-Dijkstra

```
1: function BELLMAN-FORD-DIJKSTRA(G, s)
2:     d_v ← ∞ for all v ∈ V; d_s ← 0
3:     Q ← PriorityQueue({d_s : s})
4:     while Q ≠ ∅ do
5:         A ← ∅
       // Dijkstra phase on the nonnegative edges
6:         while Q ≠ ∅ do
7:             pop u ∈ Q with minimal d_u; A ← A ∪ {u}
8:             for each (u, v) ∈ E with w(u, v) ≥ 0 do
9:                 if d_u + w(u, v) < d_v then
10:                    d_v ← d_u + w(u, v); add/update (d_v : v) in Q
11:                end if
12:            end for
13:        end while
       // Bellman-Ford phase on the negative edges
14:        for each v ∈ A and edge (v, x) with w(v, x) < 0 do
15:            if d_v + w(v, x) < d_x then
16:                d_x ← d_v + w(v, x); add (d_x : x) to Q
17:            end if
18:        end for
19:    end while
20: end function
```

To analyze it, let $d_i(v)$ be the length of a shortest path to v with at most i negative edges, and $d_i^{+1}(v)$ the length of a shortest path with at most i negative edges followed possibly by one last negative edge.

Claim 1. Invariants, by induction on the iteration i of the outer loop:

- At the end of the Dijkstra phase of iteration i: $d_v = d_i(v)$, and $A = \{v | d_i(v) < d_{i-1}(v)\}$ (for $i = 0$: $A = \{v | d_0(v) < \infty\}$).
- At the end of the Bellman-Ford phase of iteration i: $d_v = d_i^{+1}(v)$, and $Q = \left\{v | d_i^{+1}(v) < d_i(v)\right\}$.

Claim 2 (running time). Let A_i be the set A at the end of iteration i, and let $\eta(v)$ be the minimum number of negative edges on a shortest path to v. Iteration i takes $\sum_{v\in A_i}(\deg(v)+\log n)$ time, so the total running time is

$$O\Big(\sum_i \sum_{v\in A_i}(\deg(v)+\log n)\Big) = O\Big(\sum_v (\deg(v)+\log n)(\eta(v)+1)\Big).$$

A vertex enters A_i only if its distance improves in iteration i, which happens at most $\eta(v)+1$ times.
We will use the algorithm with early stopping: BFD(G,V,t) starts with $d_v \leftarrow 0$ for all v (all vertices in Q, as from a super-source) and runs the outer loop at most t times ($j \leftarrow 0$; while $Q \neq \emptyset$ and $j \le t$; $j \leftarrow j+1$). Its running time is $O(\text{Dijkstra}(G)\cdot t) = O\big((m+n\log n)\cdot t\big)$, and it computes correctly every distance realized by a path with at most t negative edges.

1.3 Potentials

For $\phi : V \to \mathbb{R}$ let $w_\phi(u,v) = w(u,v)+\phi(u)-\phi(v)$. On every path from a to b the weight changes by exactly $\phi(a)-\phi(b)$, so shortest paths are the same under w and w_ϕ, and cycles keep their weight.

- There is ϕ with $w_\phi(e)\ge 0$ for all e iff there is no negative cycle (that is, iff shortest paths are well defined).
- Given such ϕ, we can run Dijkstra on w_ϕ and compute the shortest paths.
- Given $\text{dist}(s,\cdot)$ from a super-source, $\phi(v)=\text{dist}(s,v)$ satisfies $w_\phi(u,v)=w(u,v)+\phi(u)-\phi(v)\ge 0$ (triangle inequality).

So computing a valid potential is equivalent to computing SSSP, up to one run of Dijkstra.

1.4 Restricted graphs

Definition 1. A graph is restricted if its weights are integers ≥ -1 and every cycle has mean weight at least 1, that is, $w(C)\ge |C|$ for every cycle C. In a restricted graph we may assume all weights are $< n-1$.

Theorem 3 (Restricted SSSP)**.** *In a restricted graph G we can compute a potential ϕ with $w_\phi(u,v)\ge 0$ for all $(u,v)\in E$ in expected time $T_r(m,n)=O(\text{Dijkstra}(G)\cdot\log^5 n)$.*

1.5 Scaling: from the general case to restricted graphs

Assume now that G has no negative cycles and $w(e) > -3W$.

Lemma 1. *There is an algorithm* Scale(G) *that computes a potential ϕ such that in G_ϕ, $w(e) > -2W$. It runs in expected time $O(T_r(m,n))$.*

Proof. Construct H on the same vertices and edges with $w_H(e)=\lceil w_G(e)/W\rceil+1$. H is restricted: every weight is an integer and $w_H(e)\ge\lceil(-3W+1)/W\rceil+1\ge -1$; and for every cycle C (which is nonnegative in G),

$$\sum_{e\in C}\left(\left\lceil\frac{w(e)}{W}\right\rceil+1\right)\ge |C|+\sum_{e\in C}\frac{w(e)}{W}\ge |C|.$$

The +1 buys the cycle slack. Compute (Restricted SSSP) a potential ϕ_H with $w_H(u,v) + \phi_H(u) - \phi_H(v) \geq 0$ and return $\phi(v) = W \cdot \phi_H(v)$. Since $w_H(e) < \frac{w_G(e)}{W} + 2$ we have $w_G(e) > W \cdot (w_H(e) - 2)$, hence

$$w_{G_\phi}(u,v) = w_G(u,v) + W\phi_H(u) - W\phi_H(v) > -2W + W\big(w_H(u,v) + \phi_H(u) - \phi_H(v)\big) \geq -2W.$$

■

Algorithm 4 SSSP for G without negative cycles, $w(e) > -3W$

```
function SSSP(G)
    multiply all weights by 4n
    L ← O(log nW); G_0 ← G
    for i = 0 to L − 1 do
        φ ← Scale(G_i); G_{i+1} ← (G_i)_φ
    end for
    round up to 0 all negative weights in G_L
    run Dijkstra on G_L
end function
```

Claim 3 (correctness). The algorithm is correct.

Proof. Each call to Scale reduces W (initially $O(nW)$ after the multiplication) by a factor of about $\frac{2}{3}$ ($-2W \geq -3W'$ for $W' \approx \frac{2}{3}W$), until $W = 2$, i.e. all weights are > -6; the last iteration makes the weights > -4 (but W stays 2). Shortest paths in G_L are the same as in G. Zeroing out the negative weights can increase the weight of a non-shortest path by at most $3n$, but after multiplying by $4n$ the difference between the shortest and the second-shortest path is at least $4n$. So a shortest path in the rounded graph is a shortest path in G. ■

Running time: $O(T_r(m,n) \log nW) = O(\text{Dijkstra}(G) \cdot \log^5 n \cdot \log nW)$.

1.6 Directed low-diameter decomposition

Theorem 4 (LDD). *Let $G = (V, E, w)$, $w : E \to \mathbb{Z}_{\geq 0}$, be a directed nonnegatively-weighted graph. There is a randomized algorithm that computes a set of edges E^- such that*

- *every strongly connected component of $G \setminus E^-$ has weak diameter $\leq d$;*
- *$e \in E^-$ with probability at most $O\Big(\log^2 n \cdot \frac{w(e)}{d} + \frac{1}{n^8}\Big)$.*

Weak diameter means distances are measured in all of G (paths may leave the component); strong diameter would measure them inside the component, which is larger, so weak diameter is the easier requirement. The point of the second property is that short edges are rarely cut, and long edges are cut with probability proportional to their weight. LDD in undirected graphs was known before; the twist of these algorithms is a directed version, and its use for restricted graphs.

2 Restricted SSSP

Topologically sort the strongly connected components $(S_1, \ldots, S_k)$ of $(V, E \setminus E^-, w)$. We may assume that the edges $(u, v) \in E^-$ go backwards: $u \in S_j$, $v \in S_i$ with $j > i$ (edges between components that go forward are not cut).
Notation: $G_{\geq 0}$ is G with negative weights replaced by 0, $G[S]$ is the subgraph induced by S, and $E^-_{\ell,r}$ is the set of edges between different components of $\mathcal{S}_{\ell,r}$.

Algorithm 5 Restricted SSSP

```
1: function RestrictedSSSP(G)
2:     L ← ⌈log 2n²⌉, R ← (2 + c) log n, t ← 4C log² n + 1
3:     Φ_0 ← {0}
4:     for ℓ ← 1, ..., L do
5:         for r ← 1, ..., R do
6:             S_{ℓ,r} ← (S_1, ..., S_k) = LDD(G_{≥0}, 2^ℓ)
7:             for i ∈ [k] and φ ∈ Φ_{ℓ−1} do
8:                 d_φ ← BFD(G_φ[S_i], S_i, t)
9:             end for
10:            φ^i(v) ← min_{φ∈Φ_{ℓ−1}} d_φ(v) for v ∈ S_i
11:            φ_{ℓ,r}(v) ← φ^i(v) − i · n for v ∈ S_i
12:        end for
13:        Φ_ℓ ← {φ_{ℓ,r} | r ∈ [R]}
14:    end for
15:    return φ_{L,1}
16: end function
```

Think of a table with rows $\ell = 1, \ldots, L$ and columns $r = 1, \ldots, R$; each cell stores one LDD $\mathcal{S}_{\ell,r}$ and one potential $\phi_{\ell,r}$. Within a row the repetitions are independent, and the rows are computed bottom-up: cell (ℓ, r) runs BFD inside its components with every potential of row $\ell - 1$, and takes the pointwise minimum.

2.1 The crucial property of the LDD

Claim 4. Let S be a component of $\mathrm{LDD}(G_{\geq 0}, d)$ and let P be a shortest path realizing $d(S, v)$ in $G[S]$ (from the super-source of S). Then $|P| \leq d$ and $w_{\geq 0}(P) \leq d$.

Proof. Since S has weak diameter $\leq d$ in $G_{\geq 0}$, there is a return path Q (from the end of P back to its start) with $w_{\geq 0}(Q) \leq d$. Then $w(P+Q) \leq d$, since $w(P) \leq 0$. By the cycle condition of restricted graphs, $|P|+|Q| \leq w(P+Q) \leq d$. Finally, since $w(e) \geq -1$, $w_{\geq 0}(P) \leq w(P) + |P| \leq |P| \leq d$. ■

2.2 Correctness

Claim 5 (main invariant). For all ℓ, r and $e \in E \setminus E^-_{\ell,r}$: $w_{\phi_{\ell,r}}(e) \geq 0$.

Since $L = \lceil \log 2n^2 \rceil$, $2^L \geq 2n^2 \geq (n-1)(n-2)$, so $E^-_{L,1} = \emptyset$, and the invariant gives that $\phi_{L,1}$ is a valid potential: the algorithm is correct.
The bad event $B_{\ell,r,v}$ (for level ℓ, repetition r and vertex v): fix ℓ, r and a vertex v, let S be the component of $\mathcal{S}_{\ell,r} = \mathrm{LDD}(G_{\geq 0}, 2^\ell)$ containing v, and let P realize $d(S, v)$ in $G[S]$.

- By the crucial property, $w_{\geq 0}(P) \leq 2^\ell$.
- Fix some $\mathcal{S}_{\ell-1,r'}$. An edge $e \in P$ is in $E^-_{\ell-1,r'}$ with probability $\leq C \log^2 n \cdot \frac{w_{\geq 0}(e)}{2^{\ell-1}} + O\big(\frac{1}{n^8}\big)$.
- So the expected number of edges of P in $E^-_{\ell-1,r'}$ is $\leq C \log^2 n \cdot \frac{w_{\geq 0}(P)}{2^{\ell-1}} + O\big(\frac{1}{n^7}\big) \leq 2C \log^2 n + O\big(\frac{1}{n^7}\big)$.
- By Markov, with probability $\leq \frac{1}{2}$ the number of edges of P in $E^-_{\ell-1,r'}$ is $\geq t = 4C \log^2 n + 1$.
- The repetitions are independent, so with probability $\leq \big(\frac{1}{2}\big)^R$ this happens for all $r' \in [R]$. This is the bad event $B_{\ell,r,v}$.
- Union bound over all ℓ, r, v: some $B_{\ell,r,v}$ happens with probability $\leq n \log^2 n \big(\frac{1}{2}\big)^R \leq \frac{1}{n^c}$.

Assume from now on that no bad event happens. We prove the main invariant by induction, via the two statements

Definition 2.

$$A(\ell): \ \forall r, S_i \in \mathcal{S}_{\ell,r}, v \in S_i: \quad \phi^i(v) \ \Big(= \min_{\phi \in \Phi_{\ell-1}} d_\phi(v) \Big) = d_{G[S_i]}(S_i, v),$$

$$B(\ell): \ \forall r, e \in E \setminus E^-_{\ell,r}: \quad w_{\phi_{\ell,r}}(e) \geq 0.$$

Claim 6 (base case). $A(1)$ holds.

Proof. Let P be the path realizing $d_{G[S_i]}(S_i, v)$. In level 1 the components have weak diameter ≤ 2, so by the crucial property $|P| \leq 2$, and $d_\phi = \mathrm{BFD}(G_\phi[S_i], S_i, t)$ with $\phi = 0$ finds it. Hence $\phi^i(v) = d_\phi(v) = d_{G[S_i]}(S_i, v)$. ■

Claim 7. $A(\ell) \Rightarrow B(\ell)$.

Proof. Fix r and an edge $(u, v) \in E \setminus E^-_{\ell,r}$. If u and v are in the same $S_i \in \mathcal{S}_{\ell,r}$, this follows from $A(\ell)$ (distances satisfy the triangle inequality, and the shift by $-i \cdot n$ is the same for u and v). Otherwise $u \in S_i$ and $v \in S_j$ with $i < j$ (the edge is not cut, so it goes forward), and

$$w_{\phi_{\ell,r}}(e) = w(e) + \phi^i(u) - i \cdot n - \big(\phi^j(v) - j \cdot n\big) \geq -1 - (n-1) + 0 + (j-i) \cdot n \geq 0,$$

using $A(\ell)$: $\phi^i(u) = d_{G[S_i]}(S_i, u) \geq -(n-1)$ and $\phi^j(v) = d_{G[S_j]}(S_j, v) \leq 0$. ■

Claim 8. $B(\ell) \Rightarrow A(\ell+1)$.

Proof. Fix $S_i \in \mathcal{S}_{\ell+1,r}$ and $v \in S_i$, and let P be the path realizing $d_{G[S_i]}(S_i, v)$. Since $B_{\ell+1,r,v}$ does not happen, there is r' such that P has $\leq t = 4C \log^2 n + 1$ edges of $E^-_{\ell,r'}$. By $B(\ell)$, under $\phi_{\ell,r'} \in \Phi_\ell$ all other edges of P are nonnegative, so P has at most t negative edges in $G_{\phi_{\ell,r'}}$ and $d_\phi = \mathrm{BFD}(G_{\phi_{\ell,r'}}[S_i], S_i, t)$ finds it: $d_\phi(v) = d_{G[S_i]}(S_i, v)$. The minimum over Φ_ℓ can only be smaller, and it is never below the true distance. ■

2.3 Running time and Las Vegas

- Each BFD takes $O((m_i + n_i \log n_i) \log^2 n)$ time.
- For fixed ℓ, r: $\sum m_i \le m$ and $\sum n_i \le n$, so one BFD for ℓ, r (over all components) takes $O((m + n \log n) \log^2 n)$.
- We do $|\Phi_{\ell-1}| = \log n$ of these for each ℓ, r, in total $O((m + n \log n) \log^3 n)$.
- There are $\log^2 n$ pairs ℓ, r, so $O((m + n \log n) \log^5 n)$ time.
- Each LDD takes $O(m \log^2 n)$, so the total LDD time is $O(m \log^4 n)$.

We showed an algorithm that succeeds with high probability. Testing whether a potential is valid takes $O(m)$ time, so under the assumption that there is no negative cycle we can easily turn it into a Las Vegas algorithm: check the result and repeat if it fails.

3 Low-diameter decomposition

3.1 Undirected graphs

Algorithm 6 Undirected LDD

```
1: function LDD(G, d)
2:     E⁻ ← ∅
3:     while G is not empty do
4:         pick an arbitrary vertex v
5:         sample R_v ~ Geo(p) for p ← min{1, 20 ln n / d}
6:         if R_v > d/2 then R_v ← d/2
7:         end if
8:         E⁻ ← E⁻ ∪ ∂B(v, R_v)              ▷ makes B(v, R_v) a connected component
9:         G ← G \ B(v, R_v)
10:    end while
11:    return E⁻
12: end function
```

Every component is a ball of radius $\le \frac{d}{2}$, so its diameter is $\le d$. In the lecture we ran it on a grid with $n = 48$, $d = 100$: each round grows a random ball, removes it with its boundary edges, and continues on what is left.

Claim 9 (second property). $\Pr(e \in E^-) = O\Big(\frac{w(e) \cdot \log n}{d} + \frac{1}{n^9}\Big)$.

Proof. Consider an edge $e = (u, v)$ of weight $w(e)$ and the first ball that reaches u or v, say w.l.o.g. u. The geometric distribution is memoryless, so the probability that the ball stops within the next $w(e)$ coin tosses (and cuts e) is $1 - (1-p)^{w(e)} \le w(e) \cdot p = O\Big(\frac{w(e) \log n}{d}\Big)$.
It could also be that we truncate a radius to $\frac{d}{2}$ (and cut the edge). But $\Pr\big(\mathrm{Geo}(p) > \frac{d}{2}\big) \le (1-p)^{d/2} \le e^{-pd/2} = e^{-10 \ln n} = \frac{1}{n^{10}}$, so by a union bound $\Pr(\text{some ball is truncated}) \le \frac{1}{n^9}$. ■

3.2 Directed graphs

Now G is directed with nonnegative weights, and we need strongly connected components of weak diameter $\le d$. $B_{in}(v,r)$ and $B_{out}(v,r)$ are the in- and out-balls (vertices at distance $\le r$ to v, from v), and ∂B_{in}, ∂B_{out} are the edges entering, resp. leaving, the ball.

Algorithm 7 Directed LDD

```
function LDD(G, d)
    E⁻ ← ∅
    mark v in-light if |B_in(v, d/2)| ≤ n/2, and out-light if |B_out(v, d/2)| ≤ n/2
    while G contains a vertex v marked in/out-light do
        sample R_v ~ Geo(p) for p ← min{1, 20 ln n / d}
        if R_v > d/2 then R_v ← d/2
        end if
        E⁻ ← E⁻ ∪ ∂B_{in/out}(v, R_v) ∪ LDD(B_{in/out}(v, R_v), d)
        G ← G \ B(v, R_v)
    end while
    return E⁻
end function
```

- We may assume the graph is strongly connected (otherwise work on each component separately).
- The main differences from the undirected case: separation into in- and out-balls, and carving only balls of light vertices, using recursion.
- Since a carved ball has at most $\frac{n}{2}$ vertices, the depth of the recursion is $\log n$.

In the example from the lecture ($d = 100$, so $p = \frac{20 \ln 28}{100} \approx \frac{2}{3}$) the graph has a heavy core H and two light parts: the out-ball $B_{out}(B0, 2)$ is exactly the 8 vertices of $A \cup B$ and its leaving edges are cut, then $B_{in}(Q0, 1)$ is carved, and the recursive call on $A \cup B$ carves an in-ball around $B2$.

Claim 10. The heavy vertices that remain have weak diameter $\le d$, and are strongly connected.

Proof. Let u and v be heavy. Then $\left|B_{out}\left(u, \frac{d}{2}\right)\right| > \frac{n}{2}$ and $\left|B_{in}\left(v, \frac{d}{2}\right)\right| > \frac{n}{2}$, so $B_{out}(u) \cap B_{in}(v) \neq \emptyset$: there is w in both, and a path $u \to w \to v$ of weight $\le d$ (note that w itself may be light). A strongly connected component in a recursive call is also strongly connected at the top level, so all components have weak diameter $\le d$. ■

Claim 11 (cutting probability). $\Pr(e \in E^-) = O\left(\frac{w(e) \log^2 n}{d} + \frac{1}{n^8}\right)$.

Proof. In a single LDD the same analysis as in the undirected case gives $O\left(\frac{w(e) \log n}{d} + \frac{1}{n^9}\right)$. But now an edge participates in $\log n$ LDDs (the recursion depth), and a union bound gives the claim. ■

3.3 Running time and heavy-light classification

Carving out the balls takes $O(m + n \log n)$, since each edge is scanned by at most two Dijkstra computations. The bottleneck is classifying the vertices into heavy and light, which we do by sampling.

Algorithm 8 Heavy-light classification

sample $v_1, \dots, v_\ell$, $\ell = c \log n$ ▷ the sample S
for $i = 1, \dots, \ell$ **do**
    compute $B_{in}(v_i, \frac{d}{2})$ and $B_{out}(v_i, \frac{d}{2})$
end for
for each vertex x **do**
    $|B_{out}(x, \frac{d}{2}) \cap S| \leftarrow |\{i \in [\ell] \mid x \in B_{in}(v_i, \frac{d}{2})\}|$, and symmetrically for B_{in}
    if $\frac{1}{\ell}|B_{out}(x, \frac{d}{2}) \cap S| \leq 0.6$ **then** mark x out-light
    end if
    if $\frac{1}{\ell}|B_{in}(x, \frac{d}{2}) \cap S| \leq 0.6$ **then** mark x in-light
    end if
end for

Proof. Let $X_i = 1$ if $v_i \in B_{out}\big(x, \frac{d}{2}\big) \cap S$ and 0 otherwise, so $\Pr(X_i = 1) = \frac{|B_{out}(x,d/2)|}{n}$. Let $X_{avg} = \frac{1}{\ell} \sum X_i$; then $E[X_{avg}] = \frac{|B_{out}(x,d/2)|}{n}$, and by Hoeffding $\Pr(|X_{avg} - E[X_{avg}]| \geq \epsilon) \leq 2e^{-2\ell\epsilon^2}$. With $\epsilon = 0.1$: $\leq 2e^{-0.02\ell} \leq \frac{1}{n^{10}}$ for $\ell = c \log n$ and c large enough, so with probability $1 - O\big(\frac{1}{n^9}\big)$ this bad event does not happen for any ball. Assume it does not: ■

- If x is neither out-light nor in-light, both ratios are > 0.6, hence $\frac{|B_{in}(x,d/2)|}{n} > 0.5$ and $\frac{|B_{out}(x,d/2)|}{n} > 0.5$, which is what we need for the heavy vertices to be strongly connected.
- If x is (say) in-light, the ratio is ≤ 0.6, hence $\frac{|B_{in}(x,d/2)|}{n} < 0.7$. So the recursion is still on components that are a constant fraction smaller, and the depth is still $O(\log n)$.

Running time: 2 Dijkstra runs per sampled vertex, $O((m + n \log n) \log n)$ per level; there are $O(\log n)$ recursive levels and the problems in each level are disjoint, so the total is $O((m + n \log n) \log^2 n)$.

4 A strongly polynomial algorithm for dense graphs (LLRZ)

Li, Li, Rao and Zhang (STOC 2026), "Shortcutting for negative-weight shortest paths", give an $O(n^{2.5})$-type strongly polynomial algorithm (no dependence on the weights). This part continues next lecture; these notes cover it up to the balancing of the searches.

4.1 Setup and main idea

We may assume every vertex has at most one negative outgoing edge: replace the negative edges $(u, v_1), (u, v_2)$ of weights $-6, -2$ by an edge (u, u') of weight -6 and edges $(u', v_1), (u', v_2)$ of weights 0 and 4. A vertex is negative ($\in N$) if it has a negative outgoing edge; denote by $k = |N|$ the number of negative vertices.

- Reduce the number of negative edges on a shortest path by a constant factor.
- If we can do it in subcubic time, then after $O(\log k)$ iterations we can finish with BFD.
- LLRZ obtain a faster algorithm by controlling the number of vertices added and using recursion.

Naive ambition: turn two negative hops into one. For negative edges (u, u') and (r, r') add the shortcut $w(u, r') = w(u, u') + d^0(u', r) + w(r, r')$, where d^0 is the distance using nonnegative edges only. There are at most k^2 candidate shortcuts, but computing their weights requires nonnegative-path distances: Dijkstra from each of the k negative-edge heads, $O(k \cdot (m + n \log n)) = \Omega(n^3)$ for dense graphs.

4.2 3-to-2 shortcutting

LLRZ use a weaker goal: replace three consecutive negative edges $(u, u'), (r, r'), (v, v')$ on a shortest path by two, with either a backward direct shortcut (u, r') of weight $w(u, u') + d^0(u', r) + w(r, r')$ or a forward direct shortcut (r, v') of weight $d^-(r, v) + w(v, v')$, where $d^-(r, \cdot)$ is the distance from r starting with the negative edge (r, r') and continuing with nonnegative edges. Computing all of these is too expensive, so we truncate the searches.

Definition 3. For each $r \in N$ with negative edge (r, r') we compute a threshold Δ_r and sets $V_{in}(r), V_{out}(r)$ such that

- $v \in V_{in}(r) \Rightarrow d^0(v, r) \le \Delta_r$, and $v \notin V_{in}(r) \Rightarrow d^0(v, r) \ge \Delta_r$;
- $v \in V_{out}(r) \Rightarrow d^-(r, v) \le -\Delta_r$, and $v \notin V_{out}(r) \Rightarrow d^-(r, v) \ge -\Delta_r$.

The direct shortcuts:

- add a direct forward shortcut (r, v') for $v \in V_{out}(r)$ such that (v, v') is negative;
- add a direct backward shortcut (u, r') for $u' \in V_{in}(r) \cup \{r\}$ such that (u, u') is negative.

The direct shortcuts are not sufficient: the path may enter $V_{in}(r)$ and leave $V_{out}(r)$ in the middle of its nonnegative parts. So we add a Steiner node $\tilde{r}$ and Steiner shortcuts:

- for all $x_1 \in V_{in}(r) \cup \{r\}$ and edges (y_1, x_1): if $w(y_1, x_1) + d^0(x_1, r) - \Delta_r \ge 0$, add $w(y_1, \tilde{r}) = w(y_1, x_1) + d^0(x_1, r) - \Delta_r$;
- for all $x_2 \in V_{out}(r) \cup \{r\}$ and edges (x_2, y_2): if $d^-(r, x_2) + w(x_2, y_2) + \Delta_r \ge 0$, add $w(\tilde{r}, y_2) = d^-(r, x_2) + w(x_2, y_2) + \Delta_r$.

Claim 12 (correctness). A shortest path with h negative edges becomes a path with $h - \lfloor h/3 \rfloor$ negative edges.

Proof. Look at three consecutive negative edges $(u, u'), (r, r'), (v, v')$ on a shortest path. If $u' \in V_{in}(r) \cup \{r\}$, use the direct backward shortcut. If $v \in V_{out}(r)$, use the direct forward shortcut. Otherwise let x_1 be the first vertex of the path in $V_{in}(r) \cup \{r\}$ and x_2 the last vertex in $V_{out}(r) \cup \{r\}$, with y_1 the vertex before x_1 and y_2 the vertex after x_2; we added $(y_1, \tilde{r})$ and $(\tilde{r}, y_2)$, both nonnegative. In every case the three negative edges are replaced by two. ■

4.3 How many shortcuts do we add?

Let $a_r = |V_{in}(r)|$ and $b_r = |V_{out}(r)|$.

- Direct shortcuts: $\le a_r + b_r + 1 = O(a_r + b_r + 1)$.
- Steiner shortcuts: $O(n(a_r + b_r + 1))$.

- Time to find these shortcuts: $O(n(a_r + b_r + 1) + (a_r + b_r + 1)\log n) = O(n(a_r + b_r + 1))$.
- So we want to minimize $O\big(nk + n\sum_{r\in N}(a_r + b_r)\big)$.

A large Δ_r makes $V_{in}(r)$ large and $V_{out}(r)$ small, and a small Δ_r the opposite. We will pick Δ_r so that

- $|a_r - b_r| \le 1$, and
- for all $u \in V_{in}(r)$ and $v \in V_{out}(r)$: $d^0(u, r) + d^-(r, v) < 0$.

Betweenness reduction: we will bound $\sum_{r\in N} a_r b_r \le \frac{kn^2}{b}$. This suffices, since $(a_r+b_r)^2 = (a_r-b_r)^2+4a_r b_r \le 1+4a_r b_r$, so

$$\sum_{r\in N}(a_r + b_r)^2 \le k + 4\sum_{r\in N} a_r b_r = O\left(\frac{kn^2}{b}\right), \qquad \sum_{r\in N}(a_r + b_r) \le \sqrt{k\sum_{r\in N}(a_r + b_r)^2} = O\left(\frac{kn}{\sqrt{b}}\right),$$

and in total $O\big(nk + n\sum_{r\in N}(a_r + b_r)\big) = O\left(\frac{kn^2}{\sqrt{b}}\right)$.
Two things remain: how to guarantee $\sum_{r\in N} a_r b_r \le \frac{kn^2}{b}$ (betweenness reduction), and how to balance the forward and backward searches to satisfy the two conditions above.

4.4 Balancing the searches

Run in tandem a backward search from r in the nonnegative subgraph, and a forward search from r starting with the negative edge (r, r') and continuing in the nonnegative subgraph, with two separate priority queues. Let β be the minimum unsettled backward key and f the minimum unsettled forward key; initially $\beta = 0$ and $f = w(r, r')$ (the keys of r and r').

Algorithm 9 Balancing the searches from r

```
β ← 0; f ← w(r, r')
loop
    if β + f ≥ 0 then stop
    end if
    settle a vertex in the backward search
    if β + f ≥ 0 then stop
    end if
    settle a vertex in the forward search
end loop
```

Setting Δ_r: if we stop after a forward step, set $\Delta_r = \beta$; if we stop after a backward step, set $\Delta_r = -f$. The settled vertices form $V_{in}(r)$ and $V_{out}(r)$. The proof that these choices satisfy the two guarantees, and the betweenness reduction, are left for the next lecture.